PROJET FIL ROUGE CI/CD · COMPTE RENDU

Journée 1 – Intégration continue

SouverainRecrut Backend : tests, SonarQube, Jenkins, Docker, Trivy, Docker Hub

Projet	SouverainRecrut – API Backend (Node.js 22, Express, TypeScript, Prisma, PostgreSQL)
Module	CI/CD – Mastère Expert en Développement – RNCP 40166
Séance	Séance 1 – Intégration continue avec Jenkins (cahier des charges, partie 3)
Dépôt	https://github.com/mtbvr/CICD_souverainrecrut – branche `dev`
Auteur	Matéo Bouvier
Date	9 novembre 2026

Sommaire

1. Objectif et résultat
2. Architecture
3. Choix de l'infrastructure
4. Lab Jenkins + SonarQube
5. Tests automatisés et couverture
6. SonarQube et Quality Gate
7. Image Docker
8. Scan de sécurité Trivy
9. Pipeline Jenkins et publication
10. Gestion des secrets
11. Difficultés rencontrées
12. Prochaine séance

1. Objectif et résultat

Mettre en place une chaîne d'intégration continue dans laquelle chaque modification du Backend sur la branche `dev` est automatiquement installée et compilée, testée avec un rapport de couverture, analysée par SonarQube et validée par un Quality Gate, conteneurisée, scannée par Trivy, puis **publiée sur Docker Hub uniquement si tous les contrôles précédents sont passés**.

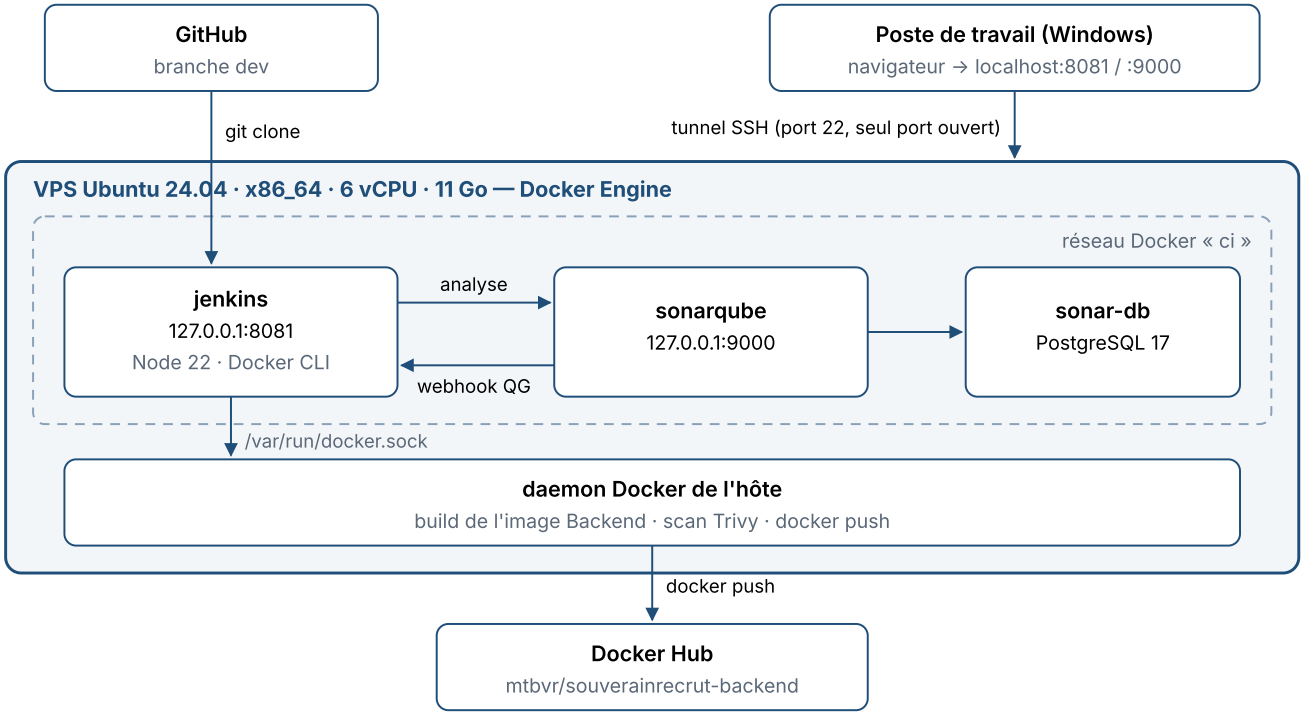
Résultat : le pipeline est complet et fonctionnel. Les builds de la journée montrent que chaque contrôle bloque réellement la publication :

Build	Tag de l'image	Résultat	Cause
#1	`1.0.0-61c559b`	Bloqué au scan Trivy	CVE critique dans `proxy-addr` (section 8)
#2	`1.0.0-563928b`	Publié	CVE corrigée ; Quality Gate par défaut *Sonar way*
#3	`1.0.0-563928b`	Bloqué au Quality Gate	Gate personnalisé *SouverainRecrut* (section 6)
#4	`1.0.0-172892d`	Publié	Code corrigé, tous les contrôles au vert
#5	`1.0.0-8b039e7`	Publié	Image durcie, 26 → 1 HIGH (section 8) : **version de référence**



Stage View Jenkins des builds #1 à #4 : #1 bloqué par Trivy, #3 bloqué par le Quality Gate, #2 et #4 publiés

2. Architecture



Composant	Rôle	Version
Jenkins	Orchestration du pipeline	LTS (JDK 21), image personnalisée
SonarQube	Analyse statique, couverture, Quality Gate	Community Build 26.11
PostgreSQL	Base de données de SonarQube	17 (Alpine)
Trivy	Scan de vulnérabilités de l'image	0.76.1 (conteneur)
GitHub / Docker Hub	Code source (branche `dev`) / registre de l'image publiée	–

3. Choix de l'infrastructure : VPS plutôt que VMware

Le formateur proposait de monter Jenkins et SonarQube dans une VM VMware (8 Go de RAM, environ 50 Go de disque). Mon poste (15,3 Go de RAM, 38,5 Go de disque libres) ne le permettait pas : le lab est hébergé sur un **VPS personnel** (Ubuntu 24.04.5 LTS, x86_64, 6 vCPU, 11 Go de RAM, 90 Go de disque libres).

- **Conforme au cahier des charges :** VMware n'est imposé que pour la partie 4 (serveur de déploiement). Le lab est décrit en Docker Compose et fonctionne à l'identique sur une VM ou un VPS (option `--vps` des scripts).
- **Architecture x86_64 vérifiée :** sur un VPS ARM, l'image serait construite en arm64 et ne tournerait pas sur la VM VMware x86 de la séance 2.
- **Cohabitation avec un site existant** (Coolify, proxy Traefik) : Jenkins déplacé sur le port 8081, Docker existant détecté et non réinstallé, 4 Go de swap ajoutés pour SonarQube.

4. Lab Jenkins + SonarQube

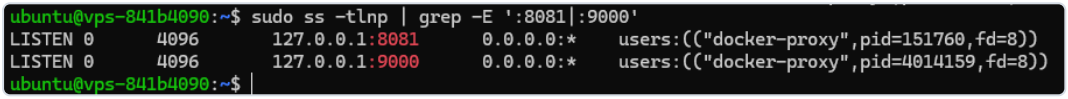
Le lab (`integration-continue/lab/`) se compose d'un `docker-compose.yml` (Jenkins, SonarQube, PostgreSQL), d'une image Jenkins personnalisée et de trois scripts : `setup-vm.sh` (préparation du serveur),

`start-lab.sh` (génération du `.env` et démarrage) et `check-lab.sh` (vérification automatique).

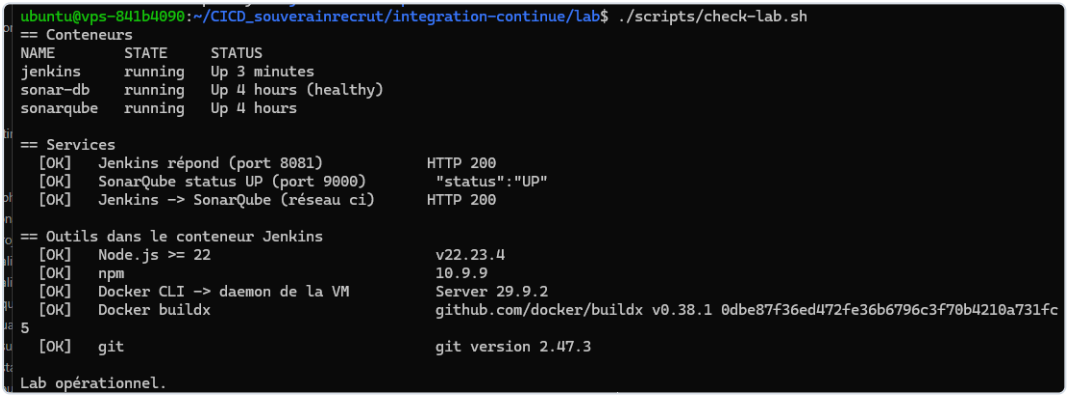
- **Image Jenkins personnalisée :** Node.js 22, Docker CLI/buildx et plugins (Pipeline, SonarQube Scanner, JUnit, Coverage, Credentials Binding…) figés dans l'image, pour une installation reproductible sans passer par l'assistant.
- **Socket Docker plutôt que Docker-in-Docker :** plus simple et cache de couches partagé. Contrepartie assumée : Jenkins dispose d'un accès équivalent root sur l'hôte.
- **PostgreSQL dédié à SonarQube** (la base H2 n'est pas supportée hors évaluation), joignable uniquement sur le réseau Docker `ci`.

Sécurité de l'accès : Jenkins et SonarQube écoutent sur `127.0.0.1` uniquement. UFW est activé (seul SSH est ouvert), mais il ne suffit pas : Docker écrit ses propres règles iptables et contourne UFW pour les ports publiés. L'accès depuis le poste se fait par tunnel SSH :

```
ssh -L 8081:localhost:8081 -L 9000:localhost:9000 ubuntu@<IP_VPS>
```



Seuls 127.0.0.1:8081 et 127.0.0.1:9000 sont en écoute



`check-lab.sh` : conteneurs démarrés, services joignables, outils présents dans Jenkins

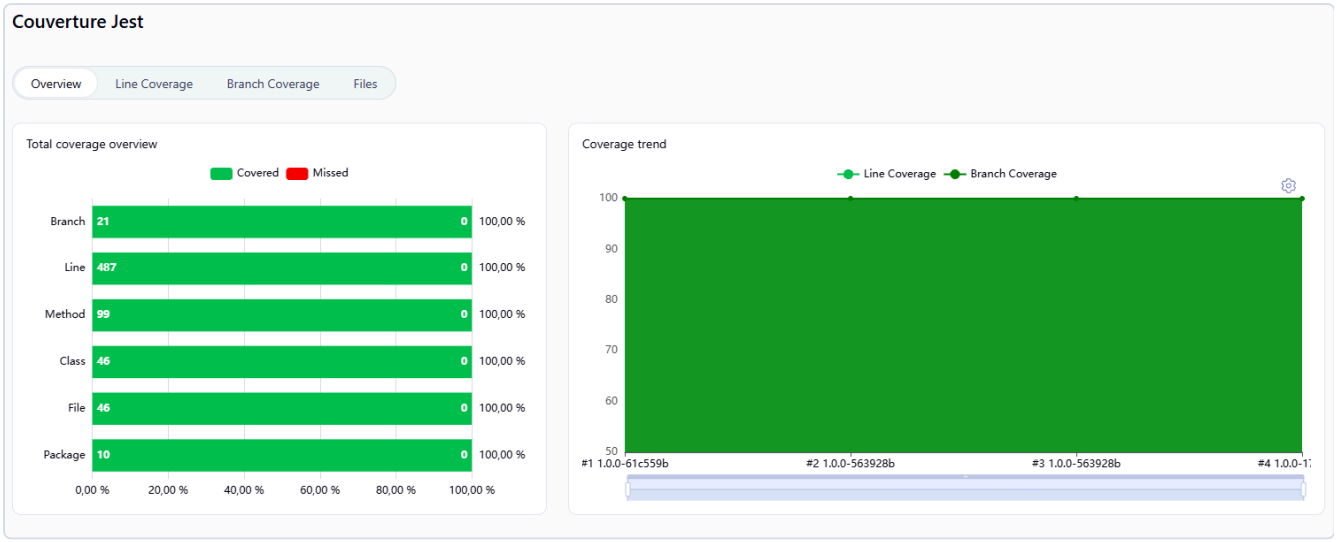
5. Tests automatisés et couverture

Le dépôt fourni ne contenait aucun test. Outils retenus : **Jest + ts-jest** (rapport lcov lu par SonarQube), **Supertest** (requêtes HTTP sur l'application Express sans démarrer de serveur), **jest-mock-extended** (mock typé du client Prisma) et **jest-junit** (résultats affichés dans Jenkins).

Stratégie : le client Prisma est remplacé par un mock, si bien que les tests tournent partout (poste, Jenkins) sans PostgreSQL, en quelques secondes et de façon déterministe. Les tests d'API traversent toute la chaîne : route → middleware JWT → contrôle des rôles → validation Zod → controller → service → repository. Limite assumée : les requêtes SQL réelles ne sont pas testées ; la séance 2 les valide au déploiement.

Domaine	Cas testés
Authentification	Login valide et invalide, `/auth/me`, token absent, mal formé, mauvaise signature ou expiré
Candidats, offres, candidatures, utilisateurs	CRUD complet, validation (400), doublons (409), ressource inexistante (404), mot de passe haché (bcrypt)
Contrôle d'accès	401 sans token, 403 pour un RECRUITER sur les routes ADMIN
Technique	Health / readiness (base UP et DOWN), Swagger, pagination, middlewares, absence de l'en-tête `X-Powered-By`

Résultats : 97 tests au vert, couverture de **100 %** (lignes, branches, fonctions, instructions). Un seuil bloquant à 80 % dans `jest.config.js` fait échouer le pipeline si la couverture baisse. Fichiers exclus, avec justification : `server.ts` (simple `app.listen`), `config/prisma.ts` (remplacé par le mock), `docs/**` et `types/**`.



Couverture publiée dans Jenkins : 100 %, stable sur les builds #1 à #4

6. SonarQube et Quality Gate

Intégration : le scanner est le paquet npm `@sonar/scan` (version figée par le lockfile, aucun Java à installer sur l'agent), configuré par `backend/sonar-project.properties`. Le token est stocké dans les credentials Jenkins. À la fin de l'analyse, SonarQube notifie Jenkins par webhook (`http://jenkins:8080/sonarqube-webhook/`) et l'étape `waitForQualityGate` récupère le verdict.

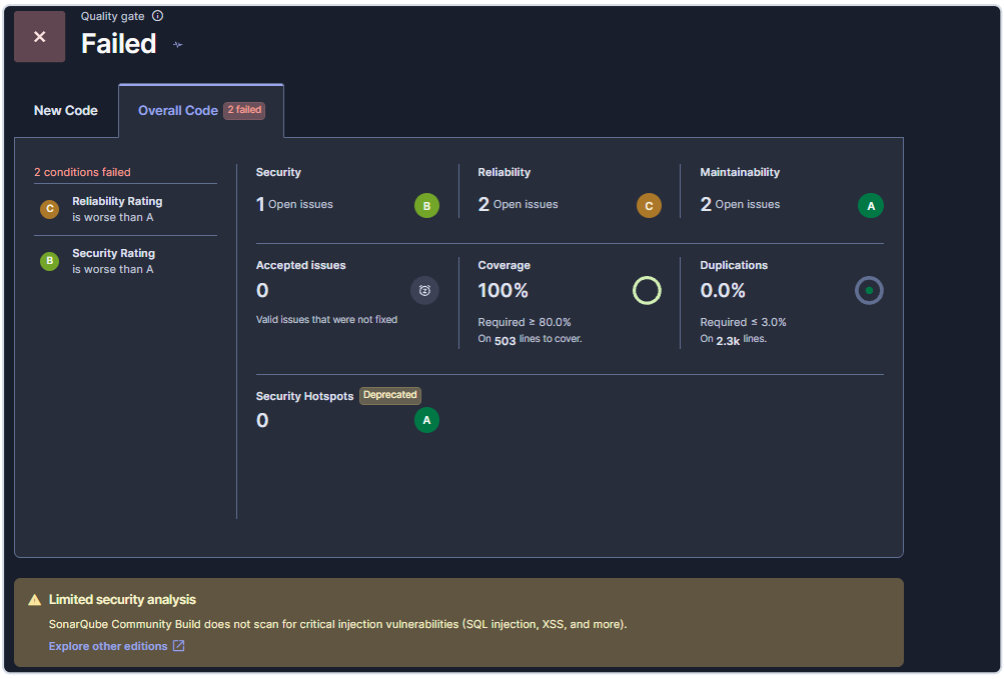
Pourquoi un Quality Gate personnalisé

Le gate par défaut *Sonar way* ne juge que le **nouveau code**. À la première analyse, rien n'est « nouveau » : les builds #1 et #2 sont passés alors que l'analyse relevait 1 problème de sécurité (note B) et 2 problèmes de fiabilité (note C). Le gate *SouverainRecrut* porte aussi sur le code global :

Portée	Condition d'échec	Justification
Code global	Couverture < 80 %	Aligné sur le seuil Jest
Code global	Duplications > 3 %	Limiter la dette de copier-coller
Code global	Fiabilité, sécurité ou maintenabilité moins bonne que A	Aucun bug, aucune vulnérabilité, dette maîtrisée
Code global	Security Hotspots revus < 100 %	Chaque point sensible est examiné
Nouveau code	Couverture < 80 %, duplications > 3 %, issues > 0	Toute évolution reste propre et testée

Démonstration : build #3 bloqué, puis corrigé

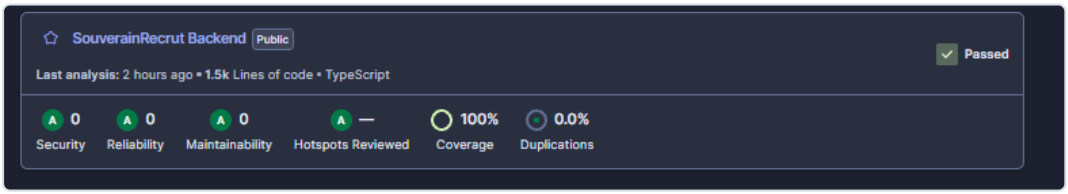
Avec le gate *SouverainRecrut*, le build #3 s'arrête à l'étape Quality Gate : le build de l'image, Trivy et la publication ne s'exécutent pas.



Build #3 : Quality Gate Failed (fiabilité C, sécurité B)

Fichier	Problème	Correction
`src/app.ts` L15	Sécurité : Express divulgue sa version (`X-Powered-By`)	`app.disable("x-powered-by")` : l'en-tête n'indique plus à un attaquant le framework à cibler. Test ajouté pour garantir son absence.
`src/utils/pagination.ts` L28, L31	Fiabilité : `parseInt` au lieu de `Number.parseInt`	Utilisation de `Number.parseInt`, explicite et cohérent avec les API ES2015.

Résultat au build #4 : **notes A partout, 0 problème, couverture 100 %, duplications 0 %, Quality Gate Passed**. Les 3 problèmes ont été fermés environ 6 heures après leur détection (MTTR mesuré par SonarQube).



Build #4 : Quality Gate SouverainRecrut Passed

7. Image Docker

Le Dockerfile fourni a été revu. Il utilise un **build multi-étapes** : l'étape de build contient TypeScript et les dépendances de développement, l'image finale ne contient que le code compilé et les dépendances de production.

Avant	Après	Raison
`node:20-alpine`	`node:22-alpine3.24`	Exigence Node 22+, base Alpine récente
`COPY . .` sans `.dockerignore`	Copies ciblées + `.dockerignore`	Ni `.env`, ni `node_modules` locaux, ni tests dans l'image
Exécution en root	`USER node`	Principe de moindre privilège
–	`HEALTHCHECK` sur `/v1/health`	État de santé visible par Docker / Compose
–	Labels OCI (version, commit, date)	Traçabilité image ↔ code source
`prisma` en devDependency	`prisma` en dependency	Exécuter les migrations depuis l'image en séance 2

Stratégie de tags : `<version>-<commit>` (ex. `1.0.0-172892d`) est **immuable** et relie l'image à la version du `package.json` et au commit Git ; c'est lui qui sert au déploiement et au rollback. Le tag `dev` est **mobile** et désigne la dernière image validée de la branche `dev`.

8. Scan de sécurité Trivy

Trivy s'exécute dans un conteneur (`aquasec/trivy:0.76.1`, version figée) avec un cache persistant de la base de vulnérabilités. Les rapports (`trivy-report.json` complet, `trivy-report.txt` HIGH/CRITICAL) sont archivés par Jenkins.

Sévérité	Traitement
CRITICAL **avec correctif disponible**	**Bloquant** : l'image n'est pas publiée
CRITICAL sans correctif, HIGH	Rapportées, non bloquantes, à suivre
MEDIUM et inférieures	Présentes dans le rapport JSON

On bloque ce qui est **actionnable** : bloquer sur des vulnérabilités sans correctif empêcherait toute livraison sans gain de sécurité.

Build #1 bloqué

Toutes les étapes précédentes étaient au vert, mais Trivy a détecté une CVE critique dans `proxy-addr` 2.0.7, une dépendance **transitive** d'Express (`express@5.2.1 → proxy-addr@2.0.7`). Express s'en sert pour déterminer l'IP réelle du client derrière un proxy : la faille permettait de fausser cette IP (journaux, limitation de débit, filtrage par adresse).

```
14:35:35  Node.js (node-pkg)
14:35:35  ==================
14:35:35  Total: 1 (CRITICAL: 1)
14:35:35
14:35:35  ┌──────────────────────────┬────────────────┬──────────┬────────┬───────────────────┬───────────────┬──────────────────────────────────────────────────────────┐
14:35:35  │         Library          │ Vulnerability  │ Severity │ Status │ Installed Version │ Fixed Version │                          Title                           │
14:35:35  ├──────────────────────────┼────────────────┼──────────┼────────┼───────────────────┼───────────────┼──────────────────────────────────────────────────────────┤
14:35:35  │ proxy-addr (package.json) │ CVE-2026-90711 │ CRITICAL │ fixed  │ 2.0.7             │ 2.0.8         │ proxy-addr: proxy-addr vulnerable to IP spoofing via │
14:35:35  │                          │                │          │        │                   │               │ IPv4-mapped IPv6 trust subnet                            │
14:35:35  │                          │                │          │        │                   │               │ https://avd.aquasec.com/nvd/cve-2026-90711               │
14:35:35  └──────────────────────────┴────────────────┴──────────┴────────┴───────────────────┴───────────────┴──────────────────────────────────────────────────────────┘
```

Build #1 : CVE-2026-90711 (CRITICAL), corrigée en 2.0.8

Correction (commit `563928b`) : la plage de versions demandée par Express accepte la 2.0.8, donc un simple `npm update proxy-addr` suffit, sans changer de version d'Express. Le build #2 passe le seuil et publie l'image. **Enseignement :** cette faille était invisible dans `package.json` ; sans scan de l'image, elle aurait été publiée puis déployée.

Durcissement de l'image : de 26 HIGH à 1 HIGH

Le scan de `1.0.0-172892d` ne montrait plus de CRITICAL, mais encore 26 vulnérabilités HIGH non bloquantes. L'analyse de leur origine a conduit à trois actions :

- **npm embarqué dans l'image Node** (10 HIGH) : npm, npx, corepack et yarn supprimés de l'image finale, où ils ne servent pas. Les migrations appellent désormais `node_modules/.bin/prisma`.
- **Dépendances transitives de l'API** (14 HIGH : `body-parser`, `fast-uri`, `js-yaml`…) : mise à jour ciblée du lockfile.
- **Dépendance de Prisma** (1 HIGH) : Prisma 6.16.2 → 6.19.3, même version majeure.

	CRITICAL	HIGH	MEDIUM	LOW
Avant (`1.0.0-172892d`)	0	26	20	3
Après durcissement	0	**1**	4	0

Risque résiduel accepté : `deepmerge-ts` (CVE-2026-40345) est épinglé en version exacte par Prisma 6 et le correctif n'existe qu'en 8.0.0. Il n'est utilisé que par la CLI Prisma au moment des migrations, pas par l'API en fonctionnement. Levée prévue avec Prisma 7.

Vérifications : 97 tests au vert (couverture 100 %), image reconstruite, migrations et démarrage de l'API testés dans la stack de déploiement. L'image durcie a ensuite été construite, scannée et publiée par le pipeline : build #5, `1.0.0-8b039e7`, qui devient la version de référence (rapports Trivy archivés dans `integration-continue/reports/`).

9. Pipeline Jenkins et publication

Le pipeline déclaratif est versionné dans `integration-continue/Jenkinsfile` (job *Pipeline script from SCM*, branche `*/dev`). Le Stage View de la section 1 en montre l'exécution.

#	Étape	Bloquant si…
1	Checkout de `dev` + calcul version / commit / tag	Dépôt inaccessible
2	`npm ci`	Lockfile incohérent
3	`prisma generate`	Schéma invalide
4	Compilation TypeScript (`tsc` strict)	Erreur de typage
5	Tests + couverture	Test en échec ou couverture < 80 %
6	Analyse SonarQube	Analyse impossible
7	Quality Gate	Gate FAILED
8	Build de l'image	Échec de build
9	Scan Trivy	CRITICAL corrigeable
10	Publication Docker Hub	Uniquement sur `dev`, après les étapes 1 à 9

Principe central : la publication est la dernière étape. Toute étape en échec arrête le pipeline, donc une image n'atteint Docker Hub que si elle a passé tous les contrôles. Autres choix : builds non concurrents, timeout de 45 min, nom du build = tag de l'image (`#N 1.0.0-<commit>`), nettoyage des images locales et de l'espace de travail en fin de build.

```
15:52:26  Image publiée : <compte>/souverainrecrut-backend:1.0.0-172892d (+ tag dev)
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] archiveArtifacts
15:52:26  Archiving artifacts
[Pipeline] sh
15:52:26  + docker image ls --format {{.Repository}}:{{.Tag}}
```

Build #4 : image publiée sur `mtbvr/souverainrecrut-backend` avec le tag immuable et le tag `dev`

Traçabilité vérifiable : le commit complet est inscrit dans le label `org.opencontainers.image.revision`.

```
docker pull mtbvr/souverainrecrut-backend:<tag>
docker inspect --format '{{ index .Config.Labels "org.opencontainers.image.revision" }}' \
  mtbvr/souverainrecrut-backend:<tag>
# -> renvoie le commit Git exact à l'origine de l'image
```

10. Gestion des secrets

Secret	Stockage	Protection
Token SonarQube	Credentials Jenkins `sonarqube-token`	Injecté uniquement pendant l'analyse, masqué dans les logs
Docker Hub	Credentials Jenkins `dockerhub-credentials`	**Access token** révocable à droits limités, jamais le mot de passe du compte
Mot de passe PostgreSQL SonarQube	`.env` sur le VPS	Généré aléatoirement, `chmod 600`, ignoré par Git

- `docker login --password-stdin` et scripts `sh` entre guillemets simples : les secrets n'apparaissent ni dans la ligne de commande ni dans les logs (pas d'interpolation Groovy).
- `.dockerignore` exclut tous les `.env` ; le `DATABASE_URL` du pipeline est factice ; aucun secret dans le dépôt Git.

11. Difficultés rencontrées

Difficulté	Cause	Solution
Poste insuffisant pour une VM de 8 Go	16 Go de RAM, 38 Go de disque libres	Lab sur un VPS, scripts compatibles VM et VPS
Port 8080 occupé sur le VPS	Traefik (Coolify) déjà en place	Port hôte de Jenkins configurable (8081)
Ports Docker exposés malgré le pare-feu	Docker contourne UFW	Écoute sur 127.0.0.1 + tunnel SSH
Webhook SonarQube refusé	Protection SSRF de SonarQube 26.x : `jenkins` résout vers une IP privée	Validation des webhooks locaux désactivée (`sonar.validateWebhooks`) : seule cible, Jenkins sur le réseau privé ; administration via tunnel SSH
Couverture des branches à 70 %	Paramètres par défaut de la pagination jamais utilisés	Tests dédiés → 100 %
Build #1 bloqué par Trivy	CVE critique dans `proxy-addr` (transitive)	Mise à jour ciblée du lockfile
Build #3 bloqué par le Quality Gate	Problèmes tolérés par *Sonar way*	Gate personnalisé sur le code global, puis correction
26 vulnérabilités HIGH dans l'image	npm embarqué + dépendances transitives	npm retiré de l'image, lockfile, Prisma 6.19.3 → 1 HIGH

12. Prochaine séance

- VM Ubuntu sur VMware (serveur de déploiement), provisionnée avec Terraform (selon l'environnement) et Ansible.
- Pipeline GitLab CI/CD : récupération de l'image depuis Docker Hub (sans reconstruction), déploiement Docker Compose + PostgreSQL, migrations Prisma, approbation manuelle et procédure de rollback vers un tag `<version>-<commit>` antérieur.